Advanced Computer-Aided VLSI System Design

AXI and AXI-Stream

Lecturer: Po-Jen Chen

Graduate Institute of Electronics Engineering, National Taiwan University



NTU GIEE



Outline

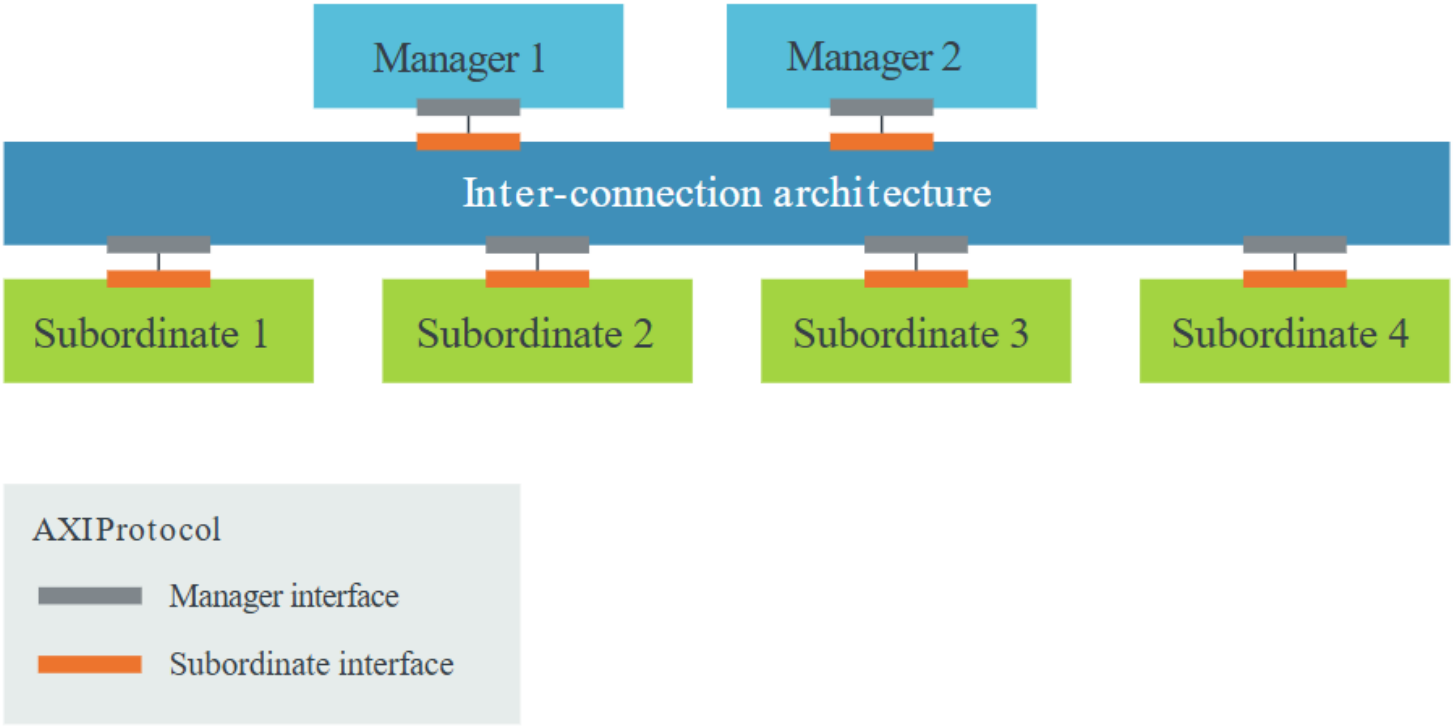
- AXI
- AXI-Stream



AXI Protocol

- Suitable for high-bandwidth and low-latency designs
- Backward-compatible with AHB and APB interfaces
- Separate address/control and data phases
- Uses burst-based transactions with only the start address
- Support for issuing multiple outstanding addresses
- Support for out-of-order transaction completion

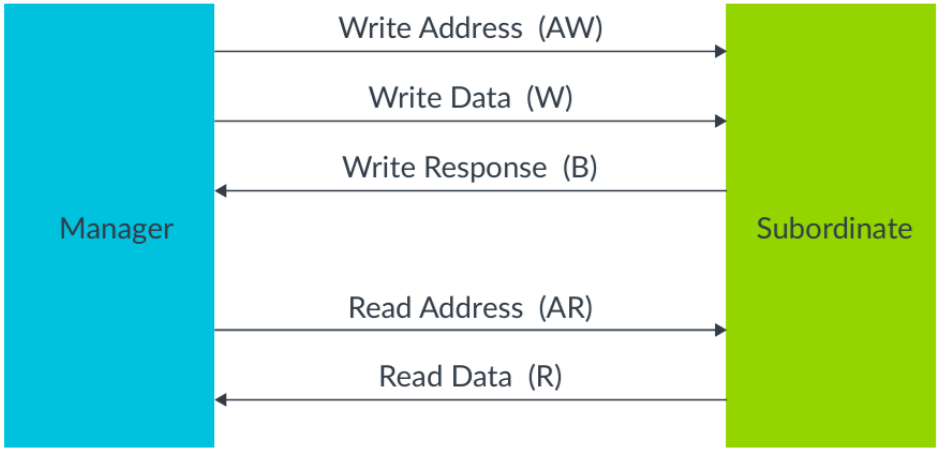
AXI Architecture



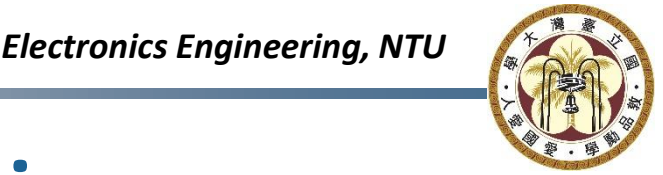


AXI Channels

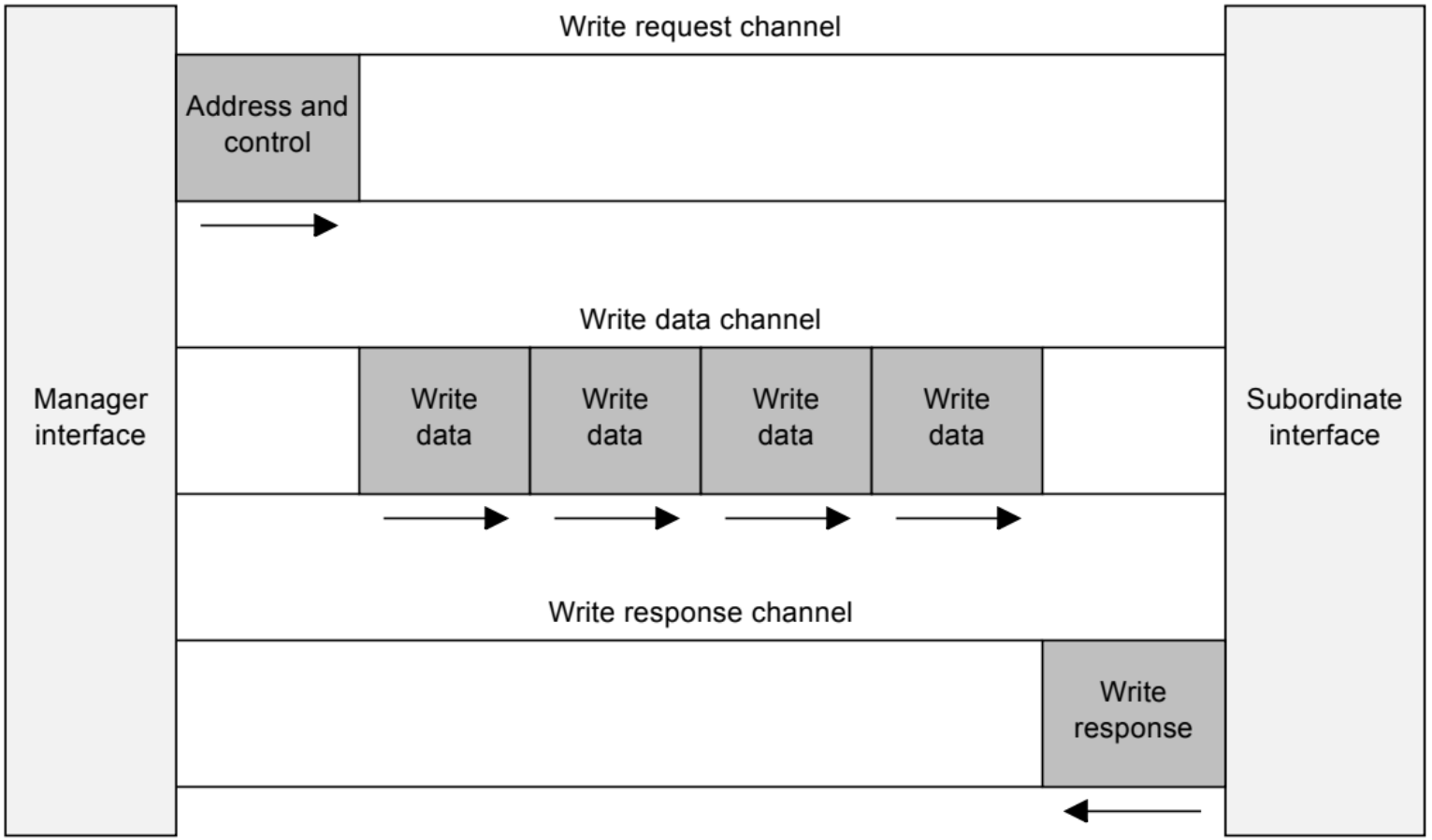
- The AXI protocol is transaction-based and defines five independent channels
  - Write request (AW)
  - Write data (W)
  - Write response (B)
  - Read request (AR)
  - Read data (R)



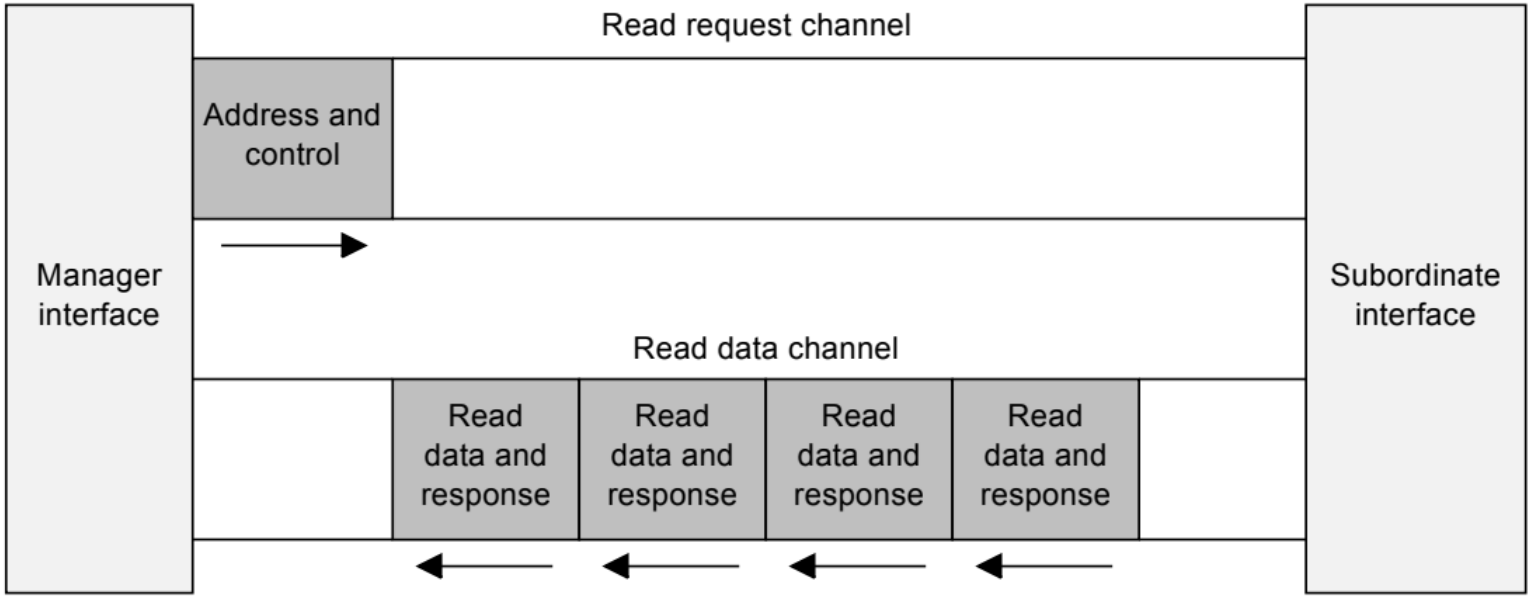
- The subordinate uses the write response channel to signal the completion of the transfer to the manager



Write Transaction

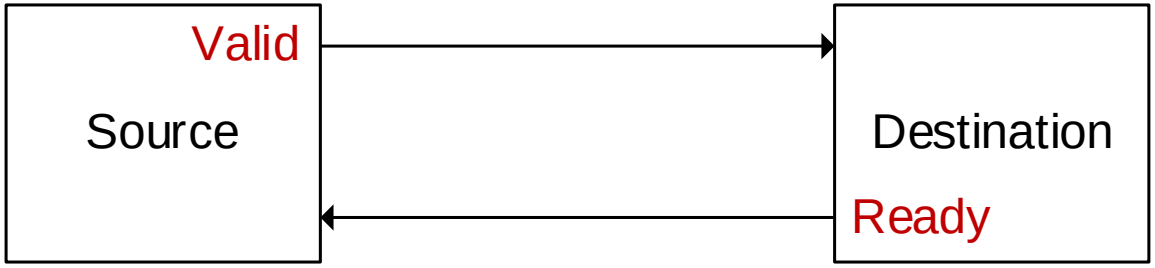


Read Transaction





Channel Handshake

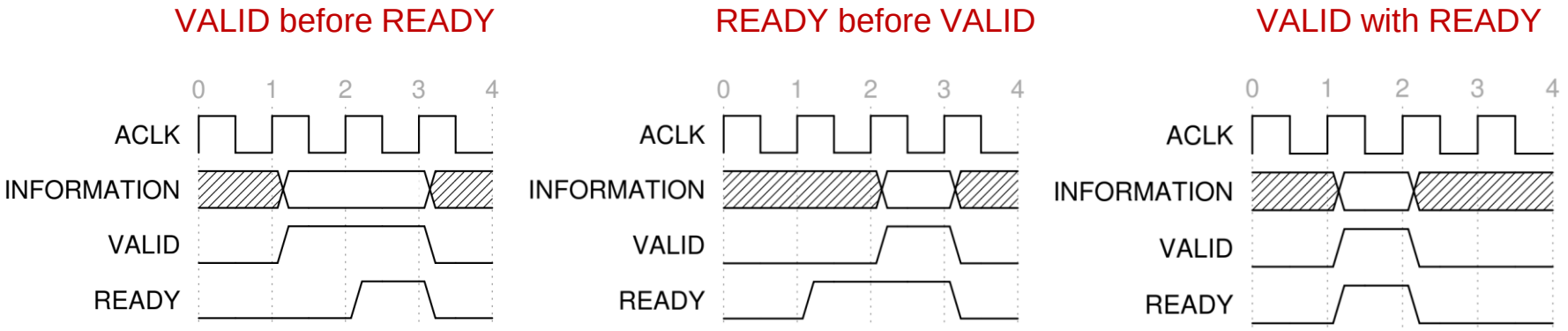


- Whether the source or destination is a manager or subordinate depends on which channel is being used
- The VALID signal indicates when valid information is available
- The READY signal indicates when the destination can accept information



Handshake: VALID and READY

- Each of the five independent channels consists of a set of information signals (address, data, and control) and **VALID** and **READY** signals



- The handshake is completed when **VALID** and **READY** signals are both high



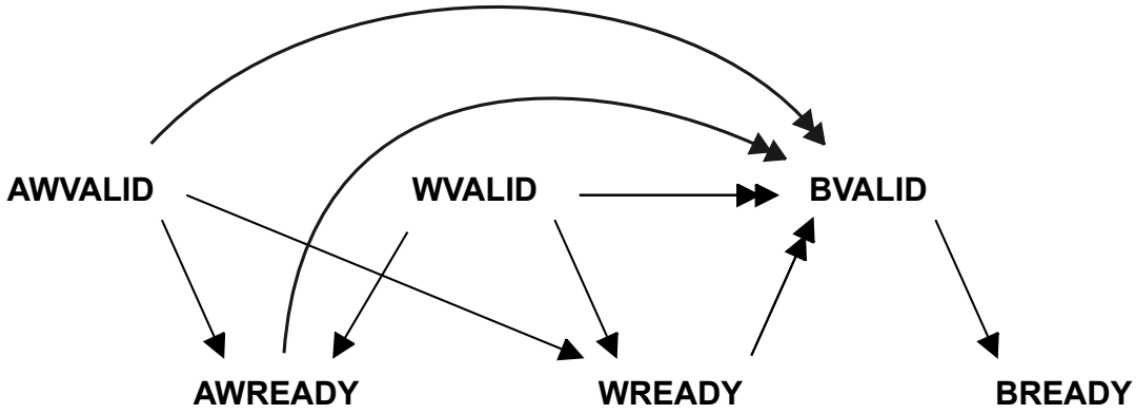
Handshake Signals

Name	Source	Description
AWVALID	Manager	Write request valid indicator
AWREADY	Subordinate	Write request ready indicator
WVALID	Manager	Write data valid indicator
WREADY	Subordinate	Write data ready indicator
WLAST	Manager	Indicates the last write data transfer of a transaction
BVALID	Subordinate	Write response valid indicator
BREADY	Manager	Write response ready indicator
ARVALID	Manager	Read request valid indicator
ARREADY	Subordinate	Read request ready indicator
RVALID	Subordinate	Read data valid indicator
RREADY	Manager	Read data ready indicator
RLAST	Subordinate	Indicates the last read data transfer of a transaction



Dependencies: Handshake Signals

Write Transaction

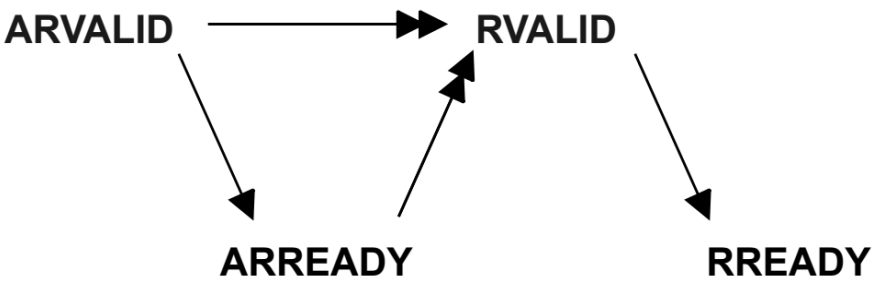


- The manager must not wait for the subordinate to assert AWREADY or WREADY before asserting AWVALID or WVALID
- The subordinate must wait for AWVALID, AWREADY, WVALID, and WREADY to be asserted before asserting BVALID
- The subordinate must not wait for the manager to assert BREADY before asserting BVALID



Dependencies: Handshake Signals

Read Transaction



- The manager must not wait for the subordinate to assert ARREADY before asserting ARVALID
- The subordinate must wait for ARVALID and ARREADY to be asserted before asserting RVALID
- The subordinate must not wait for the manager to assert RREADY before asserting RVALID



Transfer and Transaction

- A transfer is a single exchange of information, with one VALID and READY handshake

- A transaction is an entire burst of transfers
  - It includes an address transfer, one or more data transfers, and, for write sequences, a response transfer



Transaction Request

- Size Attribute
- Length Attribute
- Burst Attribute
- Transfer Address



Size Attribute

- The maximum number of bytes in each data transfer
- Read transaction
  - # of data bytes must be valid in each data transfer
- Write transaction
  - # of data byte lanes are permitted to be active (write strobe indicates which of those bytes are valid)

Name	Width	Default	Description
AWSIZE, ARSIZE	3	DATA_WIDTH/8	Indicates the maximum number of bytes in each data transfer within a transaction.



AxSIZE Encodings

- AxSIZE indicates AWSIZE and ARSIZE

AxSIZE	Label	Meaning
0b000	1	1 byte per transfer
0b001	2	2 bytes per transfer
0b010	4	4 bytes per transfer
0b011	8	8 bytes per transfer
0b100	16	16 bytes per transfer
0b101	32	32 bytes per transfer
0b110	64	64 bytes per transfer
0b111	128	128 bytes per transfer



Length Attribute

- Defines the number of data transfers in a transaction
- A manager must issue the number of write data transfers according to Length
- A subordinate must issue the number of read data transfers according to Length.

Name	Width	Default	Description
AWLEN, ARLEN	8	0x00	The total number of transfers in a transaction, encoded as: Length = AxLEN + 1.

AxLEN indicates AWLEN and ARLEN



Rules Apply to Transaction Length

- For fixed bursts, Length can be 1-16
- For incrementing bursts, Length can be 1-256
- For wrapping bursts, Length can be 2, 4, 8, or 16
- A transaction must not cross a 4KB address boundary
- Early termination of transactions is not supported
  - Neither managers nor subordinates can terminate a transaction early



Burst Attribute

- Describes how the address increments between transfers in a transaction
- Three burst types: FIXED, INCR, WRAP

Name	Width	Default	Description
AWBURST, ARBURST	2	0b01 (INCR)	Describes how the address increments between transfers in a transaction.

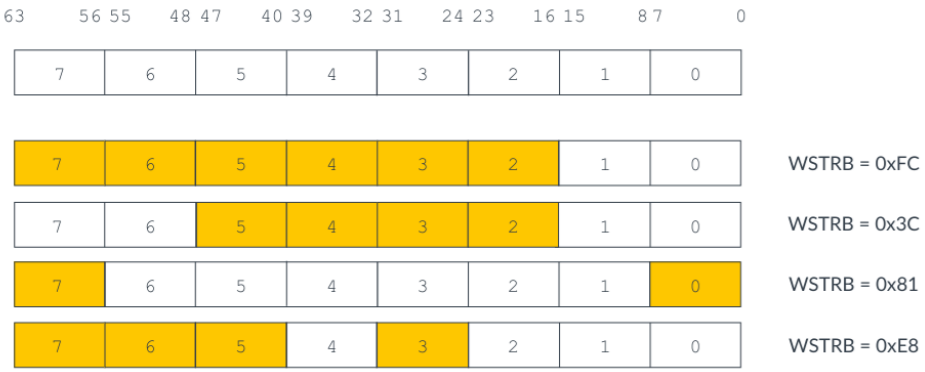
AxBURST	Label	Meaning
0b00	FIXED	Fixed burst
0b01	INCR	Incrementing burst
0b10	WRAP	Wrapping burst
0b11	RESERVED	-

AxBURST indicates AWBURST and ARBURST



FIXED

- Used for repeated accesses to the same location such as when loading or emptying a FIFO
- The address is the same for every transfer in the burst
- The actual bytes that have WSTRB asserted can differ for each transfer



- The length of the burst can be up to 16 transfers



INCR (Incrementing)

- Used for accesses to normal sequential memory
- The address for each transfer is an increment of the address for the previous transfer
- The increment value depends on the transaction size
  - For an aligned start address, the address for each transfer in a transaction with a size of 4 bytes is the previous address plus 4



WRAP (Wrapping)

- Similar to INCR except that the address wraps around to a lower address if an upper address limit is reached
  - Start address is aligned to the size of each transfer
  - Length of the burst must be 2, 4, 8, or 16 transfers
- The lowest address (wrap boundary) accessed by the transaction is the start address aligned to the total size (Size * Length) of the data to be transferred
- The address increments in the same as for an INCR burst
  - If the incremented address is ((wrap boundary) + (Size * Length)), then the address wraps to the boundary



Transfer Address

- The start address for a transaction is indicated using the AxADDR signals

AxADDR indicates AWADDR and ARADDR

Name	Width	Default	Description
AWADDR, ARADDR	ADDR_WIDTH	-	Address of first transfer in a transaction.

- ADDR_WIDTH = 1 to 64 (default: 32)

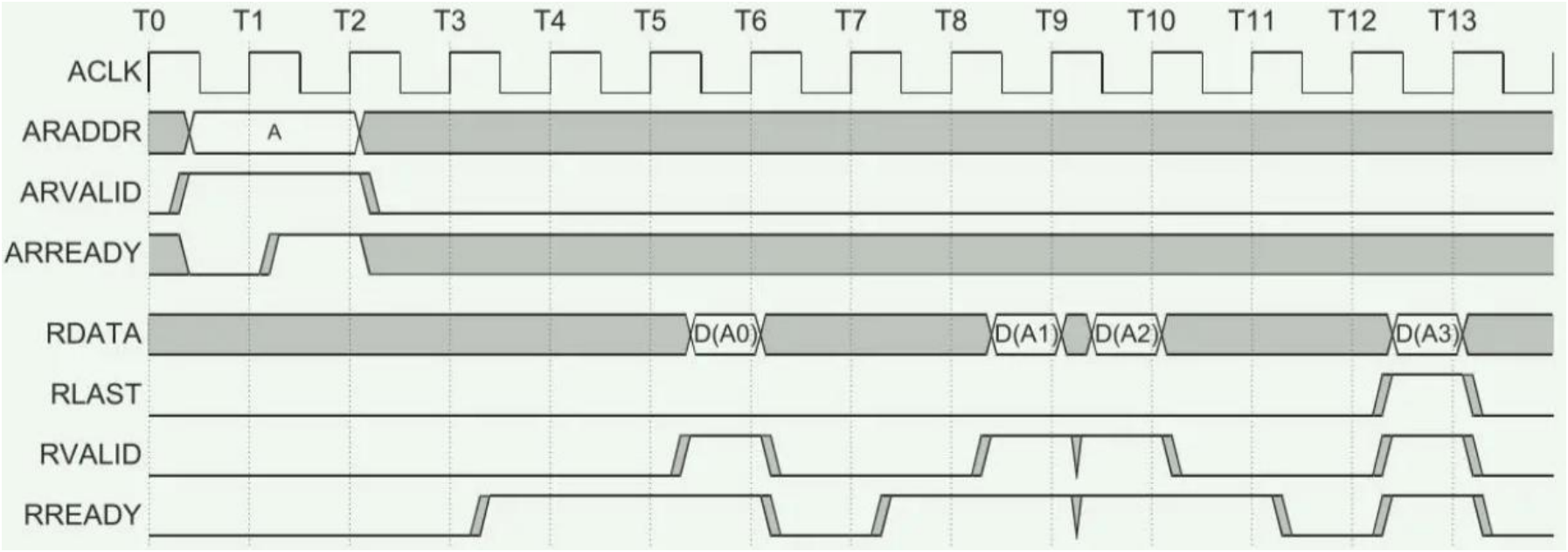


Transaction Response

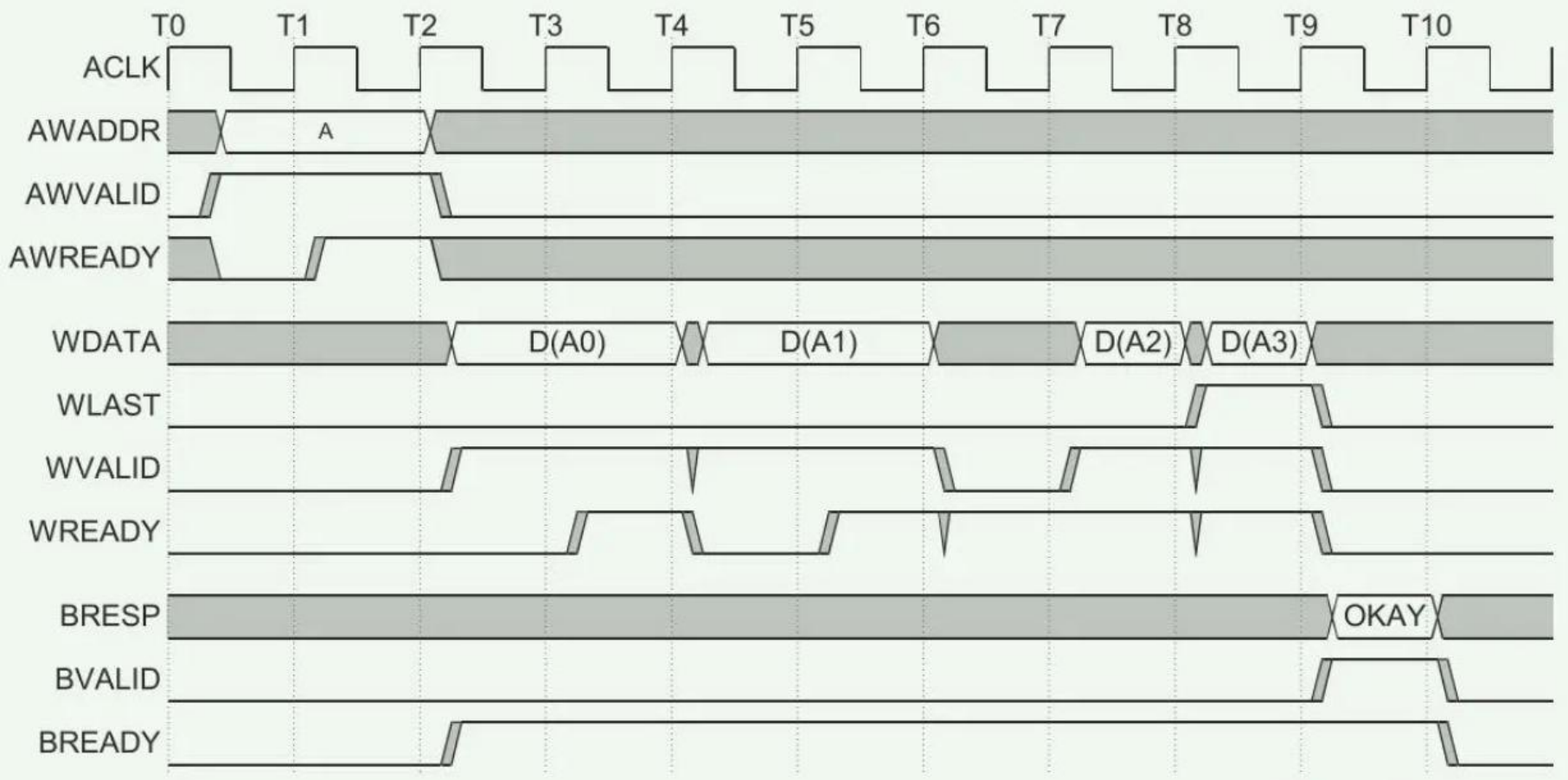
- Every AXI transaction includes one or more response transfers sent by the subordinate
- The BRESP signal indicates the result of a write transaction
- The RRESP signal on the read data channel indicates the result of each read transfer in the read transaction

Name	Default
BRESP	0b000 (OKAY)
RRESP	0b000 (OKAY)

Read Burst

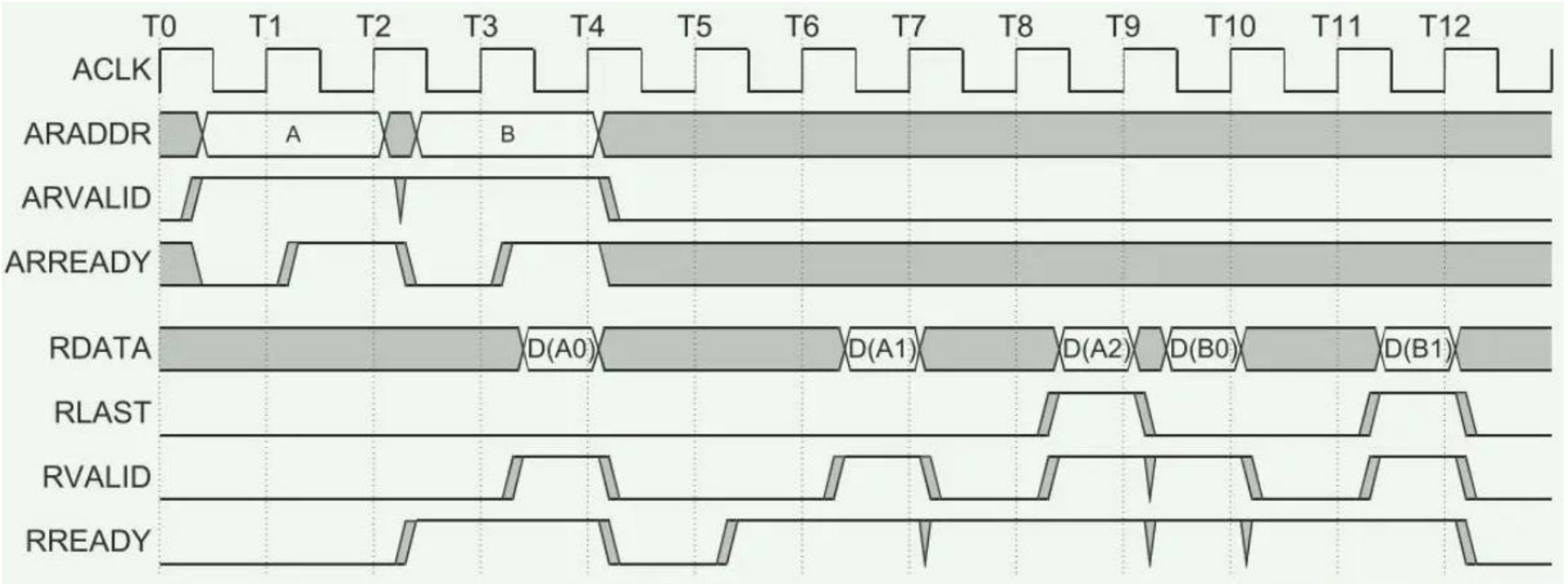

T0
T1
T2
T3
T4
T5
T6
T7
T8
T9
T10
T11
T12
T13
ACLK
ARADDR
A
ARVALID
ARREADY
RDATA
D(A0)
D(A1)
D(A2)
D(A3)
RLAST
RVALID
RREADY

Write Burst


T0
T1
T2
T3
T4
T5
T6
T7
T8
T9
T10
ACLK
AWADDR
A
AWVALID
AWREADY
WDATA
D(A0)
D(A1)
D(A2)
D(A3)
WLAST
WVALID
WREADY
BRESP
OKAY
BVALID
BREADY

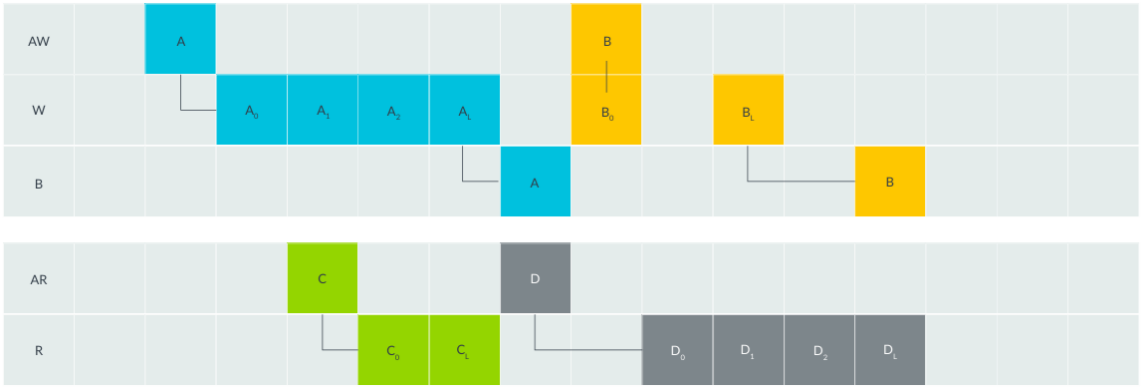
Outstanding: Overlapping Bursts

- Once the manager provides an address, it can immediately provide another one, even if the data transfer from the first address has not been completed

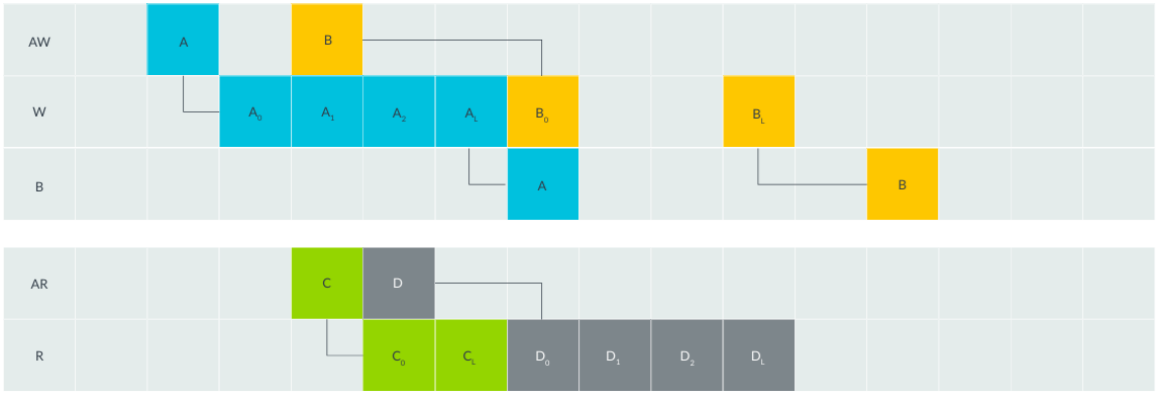


Outstanding

- Without outstanding



- Outstanding





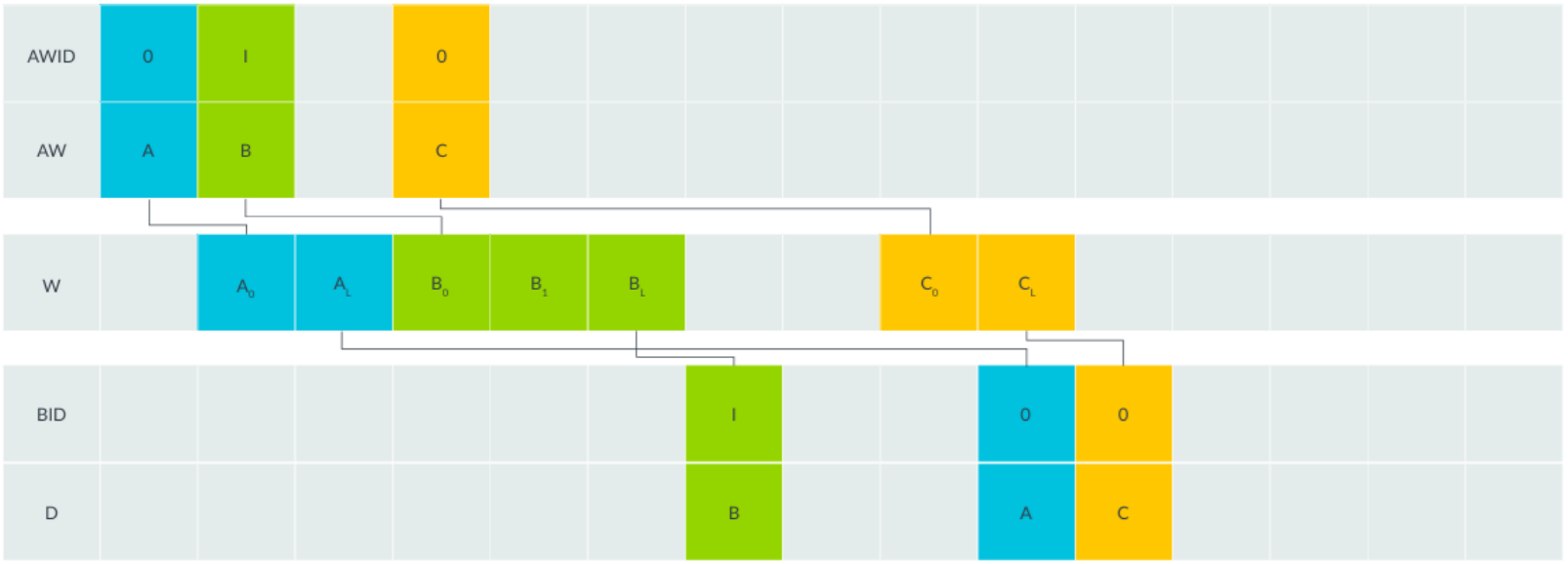
Out-of-Order Transactions

- Each channel has ID signals, which provide the capability to complete transactions out of order
  - AWID
  - WID
  - BID
  - ARID
  - RID
- All transfers must have an ID
- All transfers in a transaction must have the same ID



Out-of-Order Transactions: Write

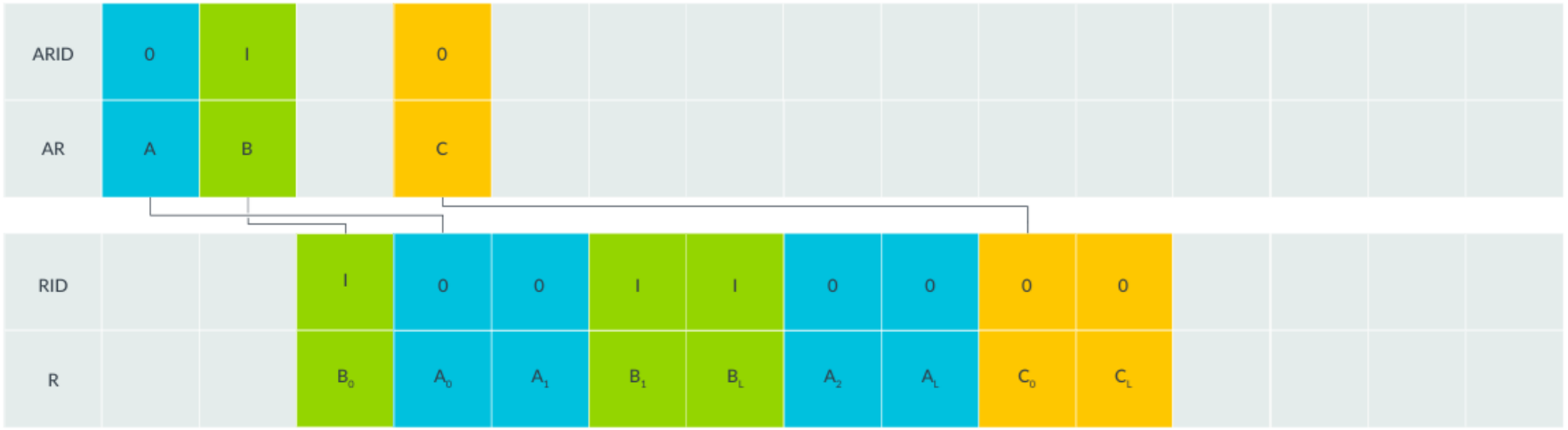
- Different ID: In-order transfer and out-of-order completion
- Same ID: In-order transfer and in-order completion





Out-of-Order Transactions: Read

- Different ID: out-of-order transfer
- Same ID: in-order transfer





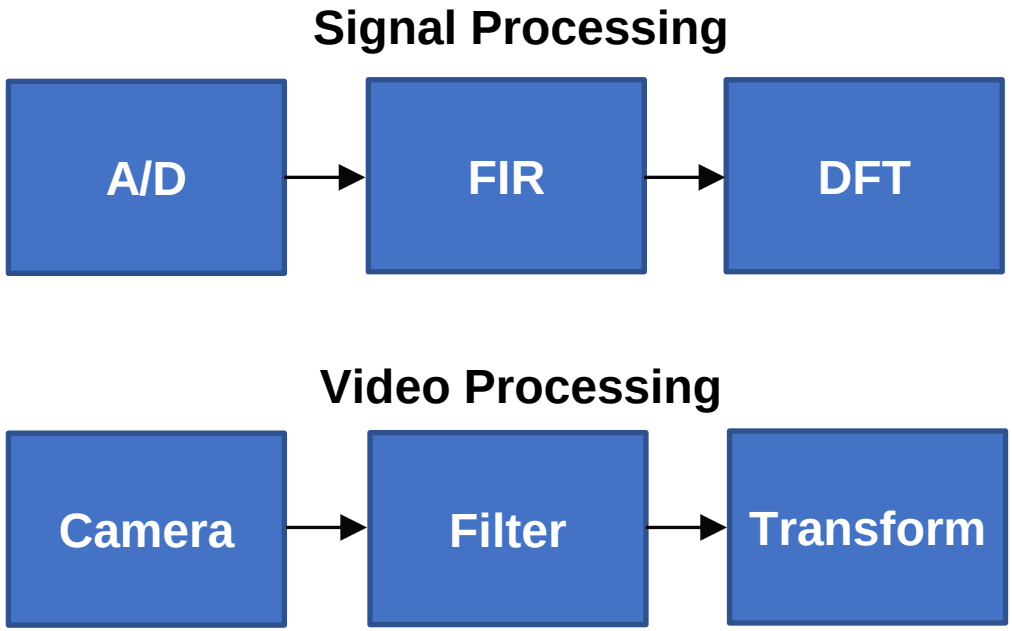
Outline

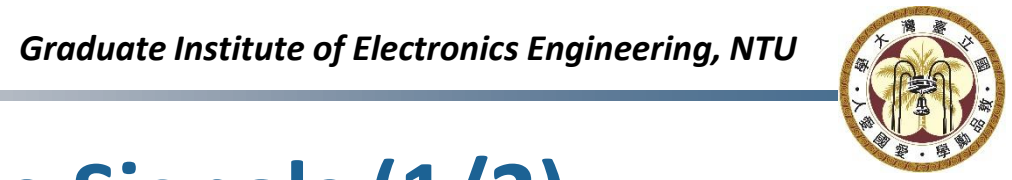
- AXI
- AXI-Stream



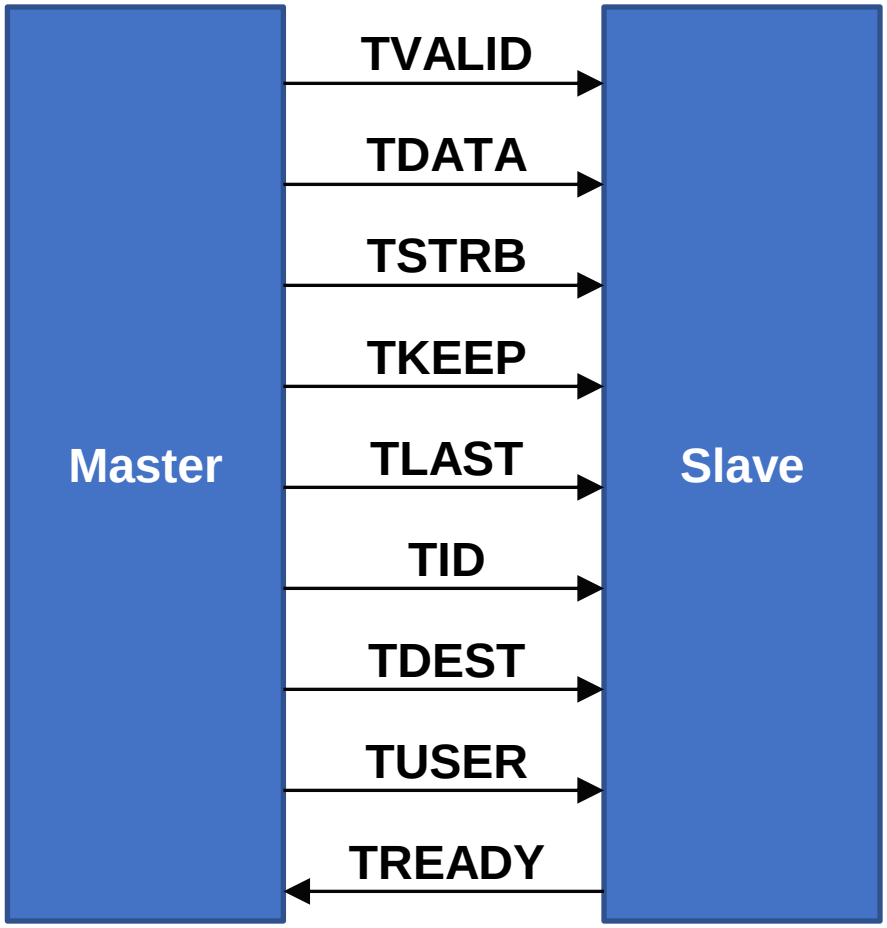
AXI-Stream

- A point-to-point protocol, connecting a single Transmitter and a single Receiver
- A non-memory-mapped protocol: No address required
- Single Direction





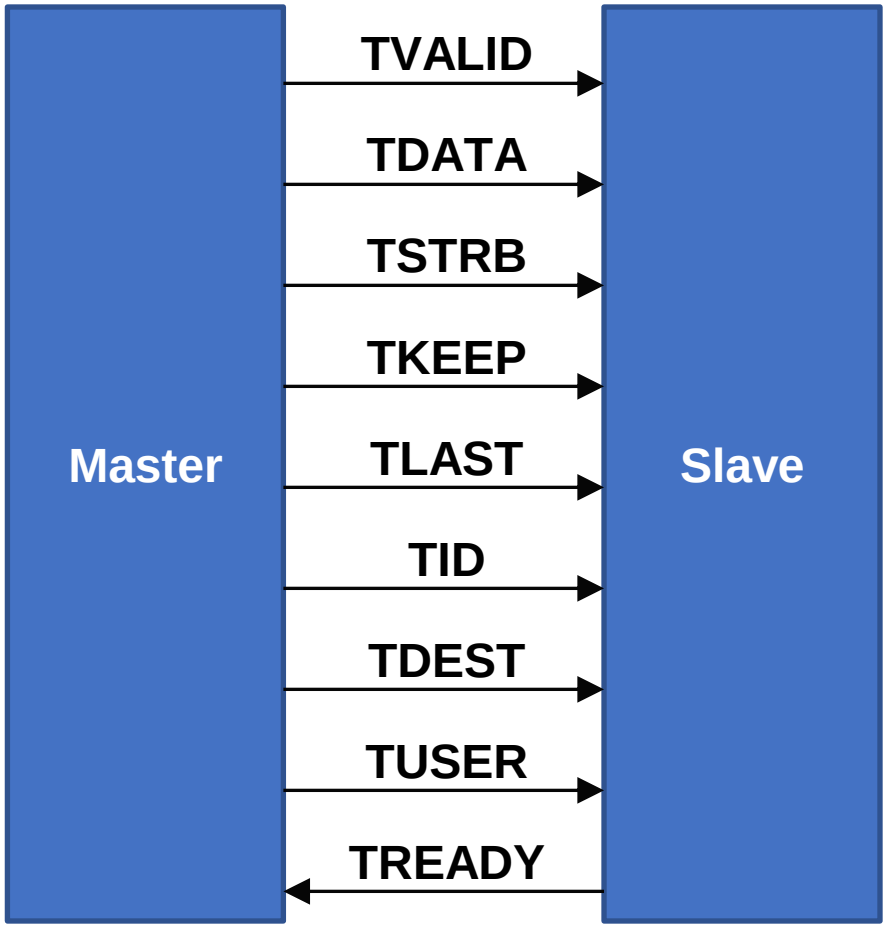
Interface Signals (1/2)



- **TVALID**: Transmitter is driving a valid transfer
- **TREADY**: Receiver can accept a transfer
- **TDATA**: Data
- **TSTRB**: Byte selection
- **TKEEP**: The byte qualifier that indicates whether content of the associated byte of TDATA is processed as part of the data stream



Interface Signals (2/2)

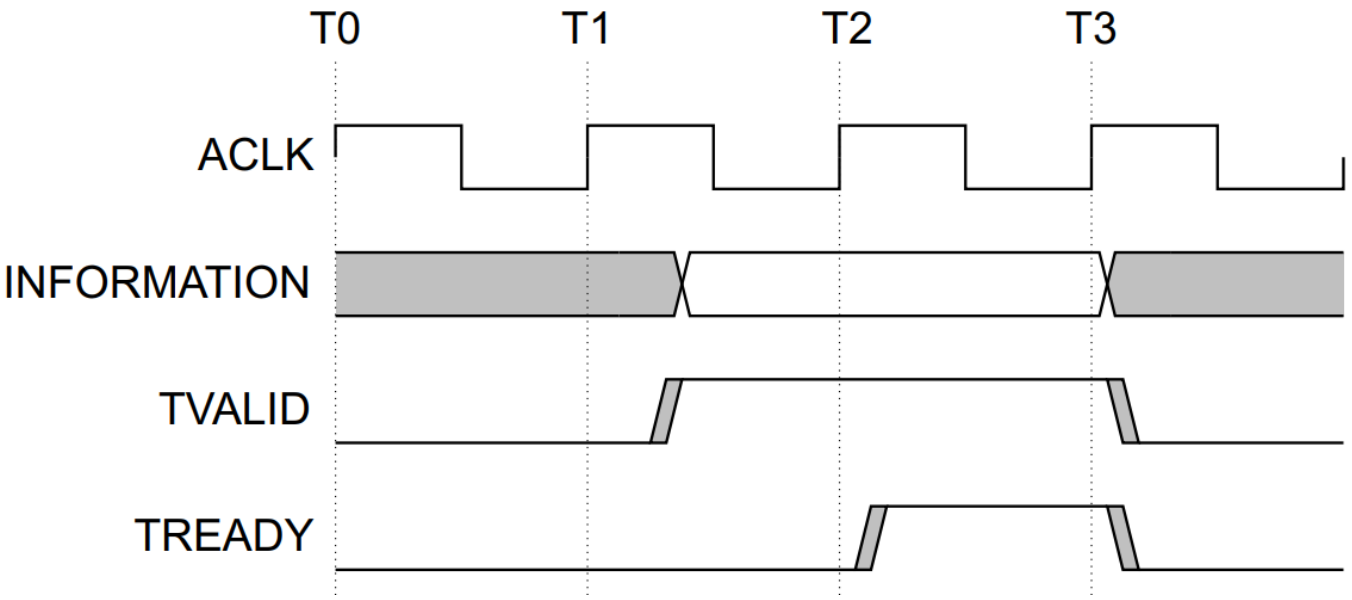


- **TLAST**: Indicates the boundary of a packet
- **TID**: Data stream identifier
- **TDEST**: Provides routing information for the data stream
- **TUSER**: User-defined information



Handshake: TVALID & TREADY

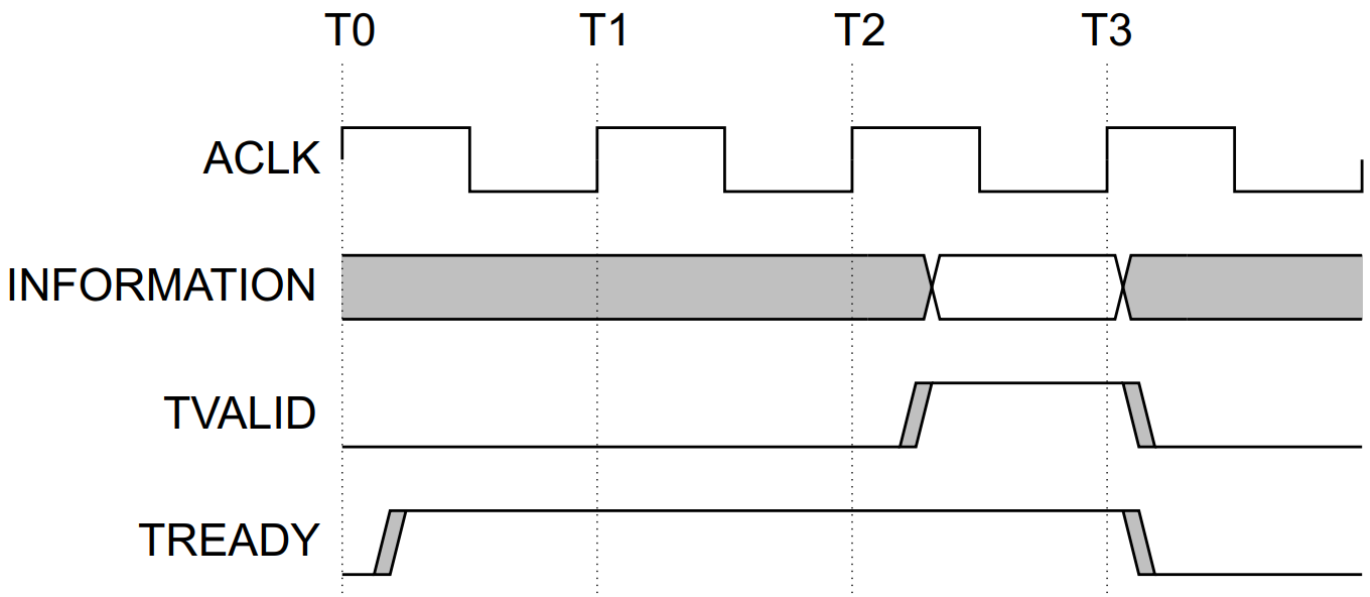
- Handshake with TVALID asserted before TREADY





Handshake: TVALID & TREADY

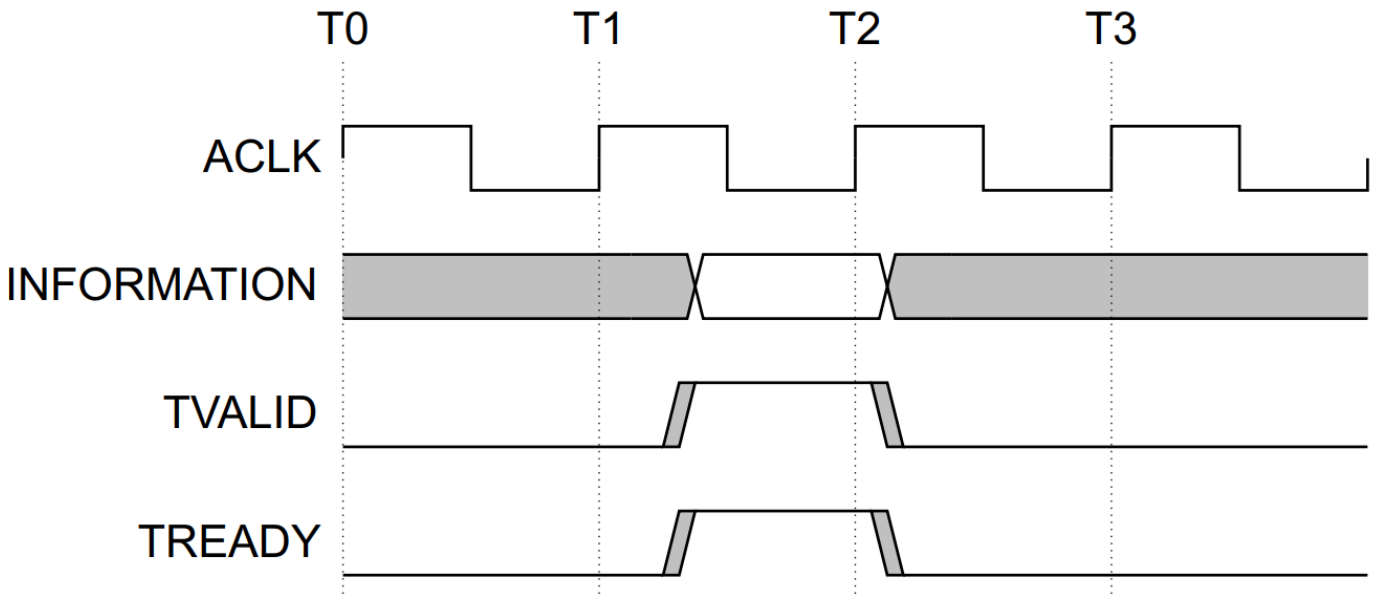
- Handshake with TREADY asserted before TVALID





Handshake: TVALID & TREADY

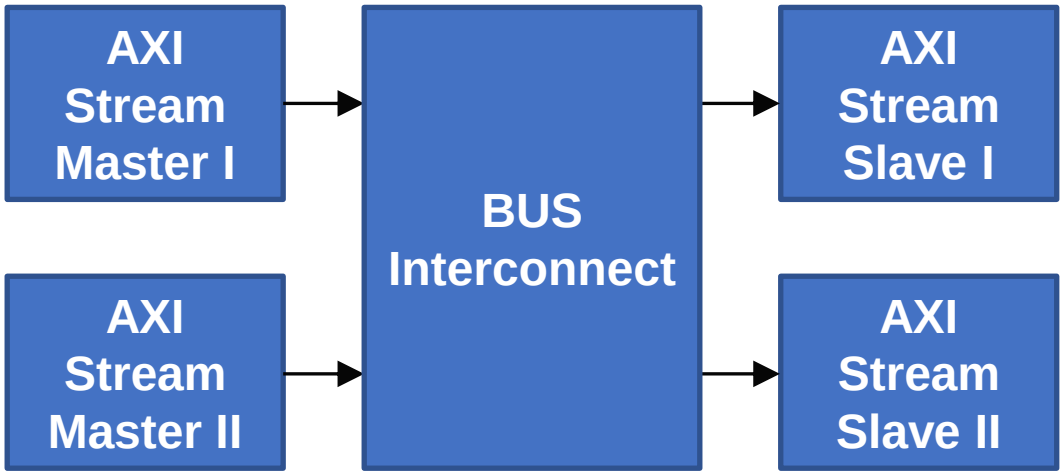
- Handshake with TVALID and TREADY asserted simultaneously





Source and Destination: TID and TDEST

- **TID**: Label the data streams from different masters
- **TDEST**: Indicate the destination of a data stream
- A common usage model is for an interconnect to generate **TDEST** information for an outgoing stream based on **TID** information of the incoming stream





Byte Types

- Data Byte
  - A byte of data containing valid information
- Position Byte
  - A byte that indicates the relative positions of data bytes within the stream
  - A placeholder that does not contain any relevant data values
- Null Byte
  - A byte that does not contain any data information or any information about the relative position of data bytes within the stream



Byte Types: TKEEP & TSTRB

- **TSTRB**: The byte qualifier that indicates whether the content of the associated byte of TDATA is valid
- **TKEEP**: The byte qualifier that indicates whether content of the associated byte of TDATA is processed

TKEEP	TSTRB	Data Type	Description
HIGH	HIGH	Data byte	The associated byte contains valid information that must be transmitted between source and destination.
HIGH	LOW	Position byte	The associated byte indicates the relative position of the data bytes in a stream, but does not contain any relevant data values.
LOW	LOW	Null byte	The associated byte does not contain information and can be removed from the stream.
LOW	HIGH	Reserved	Must not be used.



Position Byte

- The Position byte actually occupies the space, effectively serving as a 2nd byte, even though the data is invalid
- This situation usually arises when the transmission format requires a fixed structure, but a specific byte in each data set can be disregarded

Data width

D-03	D-07	Position	D-0F	D-13
Position	D-06	D-0A	Position	D-12
D-01	Position	D-09	D-0D	Position
D-00	D-04	Position	D-0C	D-10



Applications of Position Byte

- **Data Mark:** The Position Byte can be used as the beginning of a frame in video processing
- **Error Detection:** An error detection marker can be inserted after a certain number of bits to help detect errors. If the pattern is inconsistent, it indicates issues such as lost data
- **Data Group:** When four bytes of data are transmitted at once, one or two groups may be invalid. In this case, the Position Byte can be used to maintain a regular grouping of data, allowing the Slave to recognize that certain data is invalid



Example: Transmission of 16 Pixels

- Use a Position Byte to indicate the start of transmission
- Use three Null Bytes to indicate the end of transmission

